ARYAN ZAVERI

Quant Developer | Global Market Modeling | End-to-End Trading Infrastructure
LinkedIn: https://www.linkedin.com/in/aryan-zaveri0401/
GitHub: https://github.com/Aryan040101/aryan-zaveri

SUMMARY

Quant Developer with professional algo-trading/quant-dev experience across GreyOak, KIFS and Greeksoft, and a personal multi-asset research/trading infrastructure project. My strongest work is connecting research judgment with systems implementation: quantify relationships across markets, test them, replay them, and wire the surviving logic into risk-aware execution.

TECHNICAL SKILLS

Languages: Python, C++, SQL, Shell, JavaScript
Trading Systems: market data ingestion, live state, feature engineering, signal generation, strategy research, backtesting, walk-forward validation, replay, paper-fill, pre-trade risk, risk managers, order managers, order intents, execution journals, post-trade analytics
Global Modeling: oil and macro themes, Indian/global sector relationships, US/China/Japan/Korea market links, sector rotation, breadth/regime detection, dip-buy/rise-sell behavior, cross-asset correlation, options/futures relationships
Data/Infra: PostgreSQL, Redis, SQLite, JSON contracts, JSONL journals, CSV/TSV archives, Linux, AWS, Git, Docker, WebSockets, REST APIs
Metrics/Analytics: Sharpe, Sortino, profit factor, win rate, drawdown, expectancy, fees, fill rate, margin, deployed capital, capital required, side equity

WORK EXPERIENCE

GreyOak Capital - Quantitative Developer | Mar 2026 - Present

- Build quantitative research systems, market-intelligence infrastructure, market-data services and analyst-facing context-intelligence tooling.
- Work on Horizon, Compass, Context-Intelligence and Ask-GreyOak style systems connecting market context, internal data flows, dashboards, WebSocket/data contracts and research interfaces.
- Own research-to-runtime handoffs across historical archives, live state, signal outputs, runtime logs, validation reports and production-readiness workflows.
- Design data contracts and service handoffs so research outputs can be traced through dashboards, runtime systems and operational reports.

KIFS Trade Capital - Quantitative / Algorithmic Trading Developer | 2024 - Mar 2026

- Built algorithmic trading tools, broker/API workflows, intraday market-data processing, conversion/reversion research utilities and execution analytics.
- Developed intraday/options analytics around tick/depth context, price action, option-chain features, IV/OI/Greeks-style inputs, signal filters and post-market validation.
- Worked on order lifecycle mechanics, execution constraints, latency-aware design and reusable net-PnL evaluation modules for trade-quality analysis.
- Analyzed strategy behavior through charges, win/loss quality, drawdown, expectancy, execution constraints and post-market review workflows.

Greeksoft Technologies - Algorithmic Trading Intern

- Worked around broker-facing algorithmic trading platform workflows: broker connectivity, instrument mapping, market data, order placement, order status and execution flow.
- Gained practical exposure to how broker APIs, authentication, market-data subscriptions and order-state transitions fit inside an algo-trading platform.

PYTHON RESEARCH, DATA & VALIDATION SYSTEMS

Research architecture

- Built Python as the research and control-plane layer: data loading, feature tables, model training, backtesting, walk-forward validation, replay, reporting, monitoring and orchestration.
- Built pandas/numpy feature pipelines across OHLCV, returns, realized volatility, momentum, RSI, volume ratios, futures basis, IV rank, IV term slope, PCR/OI, max-pain/gamma distance, sector breadth, macro context and BTC-relative behavior.
- Worked with statistical and ML-style research components including OLS/logistic models, rolling OLS, Kalman trend logic, LightGBM-style ensembles, Random Forest/XGBoost/LightGBM model families, threshold search and temporal validation.

Validation and portfolio research

- Built no-lookahead backtesting patterns using completed-prior-data features, train/test windows, out-of-sample metrics, Sortino-style objectives, quality gates, cost/slippage assumptions and replayable reports.
- Built equity/futures research scripts for rule-family analysis, state-risk walk-forward tests, green/red side books, combined portfolio comparisons, sector caps, MTM curves and daily/monthly summaries.
- Built crypto research engines for movement/relationship candidates, weekly streaming walk-forward selection, state-risk grids, density/risk grids, stress tests, feature diagnostics and capital-accounted simulations.

Data, orchestration and monitoring

- Built options research/live-selection pipelines for dataset construction, feature-table building, model bundles, replay inference, live inference, threshold selection, Postgres result storage and monitor views.
- Built Redis/PostgreSQL Python services including stream consumers, bar/L1/L2 state writers, candle writers, signal-to-executor bridges, symbol resolution caches, freshness/slippage guards and heartbeats.
- Built FastAPI/monitor/reporting utilities for spread views, backfill healing, market briefs, operational dashboards and research-output inspection.

C++ TRADING RUNTIME & EXECUTION SYSTEMS

Runtime architecture

- Built C++ runtime components across exchange feeds, private order sockets, live matrix engines, signal generators, risk governors, order executors, exit managers and dashboard adapters.
- Used Boost.Asio/Beast/OpenSSL for networked socket and HTTPS-style flows; nlohmann::json for contracts and journals; CMake for

build targets; threads, atomics, mutexes and condition variables for live state coordination.
- Worked with hiredis/Redis hot state and libpq/PostgreSQL-style persistence across order streams, position state, market data, heartbeats, retention repair and durable analytics outputs.

Execution and risk path
- Implemented order preflight checks around account state, instrument cache, leverage/margin context, Redis top-of-book quotes, entry-price drift, quote freshness and duplicate-position protection.
- Built smart-execution flows that consume preflight output, risk contracts and private socket state, then produce exchange/order journals, smart-execution journals, fill/reject state and replayable snapshots.
- Built risk-governor style flows that read actionable signals, strategy plans, paper/live positions, account probes and risk contracts, then emit risk decisions and order intents through JSON outputs.

Data, replay and deployment
- Built C++ tooling for live matrix scoring, feature parity, signal replay parity, exit replay parity, paper-fill replay, live-folder backtests, strategy contract promotion and weekly rule/model tooling.
- Structured runtime deployment around systemd services for market sockets, signal bridges, risk managers, order executors, position trackers, Redis writers, PostgreSQL writers, memory guards and crypto live services.

PROJECT EXPERIENCE - VAJRA PERSONAL RESEARCH/TRADING INFRASTRUCTURE

Market framework
- Built Vajra around the idea that markets do not move in isolation: Indian sectors, global indices, US/China/Japan/Korea market context, crude oil, FX, macro variables, options/futures structure and crypto risk appetite all feed the research process.
- Focused on quantifying relationships instead of relying on narrative: correlations, lead/lag behavior, breadth, sector themes, volatility regimes, option-chain structure, cross-asset stress, capital usage, execution quality and replay feedback.
- Designed models to compare market state across live execution, intraday signal selection, swing/regime context and capital review, then translate the best available view into risk-aware action.
- Researched themes including dip-buy/rise-sell behavior, momentum, reversal, mean reversion, volatility/options, breadth, regime detection, sector rotation and cross-asset confirmation.

Research-to-execution architecture
- Built the full loop from market data and feature engineering to cross-market relationship analysis, signal ranking, risk decisioning, order intents, execution journals, replay/PnL analytics and model refinement.
- Separated Python research/backtesting from C++ live/runtime systems so research can iterate quickly while execution, risk and state management stay explicit and fast.
- Used JSON contracts for signals, risk decisions, order intents, position snapshots, execution events and journals; Redis for hot queues/streams/hashes/positions/heartbeats; PostgreSQL/SQLite for durable candles, instruments, Greeks, snapshots and reports.
- Built clear process boundaries between research, simulation, replay, paper/live-runtime records and post-trade analytics so each result can be interpreted in the right context.

Risk and execution systems
- Built risk around a single independent authority: strategies propose, risk decides, execution implements. Controls cover global capital, segment risk, side exposure, symbol exposure, active-order limits, quote freshness, drawdown state and hard exits.
- Worked on adaptive execution logic: spread/slippage gates, stale-quote rejection, price leaning around bid/ask/mid, replace/adjust loops, chunking, laddering, take-profit, stop-loss, trailing exits and time/hard exits.
- Designed live systems around low-latency state discipline: memory-resident working orders, Redis hot path instead of database reads in execution, queues/streams between components and batched durable writes outside the hot path.
- Modeled execution quality through preflight readiness, duplicate-intent checks, exchange gates, slippage constraints, fill/reject journals, position state and replay analytics.

Strategy and validation surface
- Researched families across momentum, reversal, mean reversion, dip-buy/rise-sell behavior, volatility/options, breadth, regime detection, sector rotation, cross-asset relationships, portfolio/risk overlays and execution-aware trading.
- Built equity/futures relationship studies using market and sector anchors, side-specific rules, state-risk overlays and rolling train/test separation.
- Built options research pipelines for index/equity call-put workflows using OI, IV, Greeks-style features, expiry, moneyness, price movement and intraday option behavior.
- Built completed-prior-data walk-forward studies, fee-aware replay, paper-fill simulations, execution-variant experiments and reports for drawdown, win rate, profit factor, Sharpe/Sortino, fees, slippage, fill rate and capital usage.

Live/replay evidence
- Built live/replay infrastructure connecting archives, live-current signal files, model artifacts, strategy contracts, risk contracts, order intents, smart-execution journals, exchange journals and socket events.
- Generated structured JSON/JSONL lifecycle logs for signal generation, candidate selection, risk decisions, order intents, exchange submission paths, exit journals, smart-execution decisions and order-stream events.
- Tracked preflight readiness, duplicate-intent checks, exchange gates, slippage constraints, laddering fields, exchange-send status, side capital, projected exposure, rejection reasons and portfolio state.

ENGINEERING FOCUS
- Build systems with clear boundaries between research, simulation, replay, paper/live-runtime records and post-trade analytics.
- Prefer simple operational contracts: JSON messages, Redis hot state, PostgreSQL durable storage, replayable journals, heartbeats and observable runtime state.
- Optimize live paths by keeping active market state and working orders in memory, using queues between components and moving durable writes outside the execution path.

EDUCATION
B.Tech Information Technology, MPSTME.